

Дополнительная лекция по Си № 9

14 мая 2025 г.

Коновалов А.В.

Написание функций с переменным числом параметров

Исторический аспект

В языке Си до первой стандартизации в 1989 году (до стандарта, который часто называют C89) функции объявлялись и определялись в следующем стиле:

```
/* Объявления функций */
int f();

/* Определения функций */
int f(a, b, p, x, y)
    int a, b;
    int *p;
    double x, y;
{
    ...тело функции...
}
```

Т.е. при объявлении функций параметры вообще не указывались, при определении функций в круглых скобках указывались только имена параметров, а их типы описывались между параметрами и составным оператором тела функции.

Для сравнения, после стандартизации в 1989 году ввели новый синтаксис, которым мы пользуемся сейчас:

```
/* Объявления функций */
int f(int a, int b, int *p, double x, double y);

/* Определения функций */
int f(int a, int b, int *p, double x, double y)
{
    ...тело функции...
}
```

До официальной стандартизации в 1989 году в качестве стандарта де-факто использовалась книга Кернигана и Ричи «Язык программирования Си» (авторов самого языка Си). Поэтому «достандартный» Си часто называют K&R Си.

Объявления и определения функций в «старом стиле» (в стиле K&R) приводят к двум последствиям:

- Если функция объявлена, то её можно вызвать с любым количеством параметров, т.к. при объявлении функции параметры вообще не указываются.
 - Следствие из этого: язык способствовал ошибкам — можно было вызвать функцию с неправильным количеством параметров, компилятор на такие ошибки не выдавал. Более того, можно было вызвать необъявленную функцию (это приводило в лучшем случае к предупреждению), подразумевалось, что функция возвращает `int`.
 - Среди системных вызовов POSIX (так называются стандартные системные функции) есть функция `open`, открывающая файл. Её можно вызывать или двумя, или с тремя параметрами в зависимости от флагов (передаваемых вторым параметром).
 - Вызов функции должен генерироваться компилятором таким образом, чтобы её можно было вызывать с любым числом параметров и свои параметры функция должна уметь найти.
- Согласно «старому» синтаксису описания типов параметров располагаются между закрывающей круглой `)` и отрывающей фигурной `{` скобками. Поэтому, в отличие от других конструкций языка Си, тело функции может быть не любым оператором, а только составным оператором, т.к. фигурная скобка означает завершение списка типов параметров.

К чему этот экскурс: функция, объявленная как `f()` — с пустыми скобками, согласно C89 (и последующими, возможно, не всеми) может принимать любое количество параметров. Для объявления (и определения) функций, которые вообще не принимают параметры, стандарт C89 утвердил следующий синтаксис:

```
double функция_без_параметров(void);
```

т.е. ключевое слово `void` внутри круглых скобок. Компилятор будет выдавать ошибку при попытке вызова такой функции с какими либо параметрами.

Некоторые стандарты оформления кода на Си предписывают объявлять функции без параметров именно с `void` между круглыми скобками.

Каким же образом компилятор генерировал код для вызова функции? (Также код и генерируется сейчас на некоторых платформах для функций с произвольным количеством параметров.)

Параметры должны передаваться через стек, причём в обратном порядке. При необходимости, параметры должны расширяться (`char, short` → `int`, `float` → `double` и т.д.).

Если параметры функции передаются в обратном порядке, то в момент вызова

на вершине стека будет находиться самый первый параметр, под ним второй и т.д. Поэтому, если параметров передано больше, чем нужно, то функция свои первые несколько параметров обязательно найдёт.

Если функция вызвана с неправильным количеством параметров (меньше, чем нужно) или с параметрами не того типа (ожидался `double`, 8 байт, получен `int`, 4 байта), то функция увидит в параметрах неопределённые значения.

Поскольку «старый» Си позволял вызывать функцию с любым количеством параметров, то программисты стали писать функции, которые принимают произвольное количество параметров: в определении указывались первые несколько обязательных параметров, к последующим обращались, взяв указатель на последний и используя адресную арифметику для обращения к содержимому стека за ним.

На большинстве архитектур стек растёт в сторону младших адресов, поэтому «дополнительные» параметры располагаются по адресам, большим, чем адрес последнего *явного* параметра.

Несколько таких функций с произвольным количеством параметров даже вошли в стандартную библиотеку языка: это семейство функций `printf` и `scanf`.

Прямое обращение к «дополнительным» параметрам существенно зависит от платформы, что делает использование непосредственной адресной арифметики непереносимым.

Стандарт C89 узаконил функции с произвольным числом параметров и предоставил переносимые средства для работы с ними.

Функции с эллипсисом (многоточием, ...) и библиотека `stdarg.h`

Для написания функции, которая принимает переменное количество параметров, после обязательных параметров нужно указать эллипсис, т.е. многоточие:

```
int sum(int count, ... );
```

При вызове таких функций типы обязательных параметров должны быть указаны правильно, типы дополнительных параметров могут быть любыми — ответственность за них лежит на программисте.

Как же к ним обращаться? Есть непереносимый, нерекомендуемый, опасный способ использовать адресную арифметику. Такое можно встретить в некоторых учебниках, но так делать не надо:

```
#include <stdio.h>
```

```
/* Функция суммирования нескольких целых чисел */
int sum(int count, ...)
{
```

```

    int *p = &count + 1;
    int res = 0;
    for (int i = 0; i < count; ++i) {
        res += *p;
        ++p;
    }
    return res;
}

int main()
{
    int res = sum(5, 1, 2, 3, 4, 5);
    printf("1+2+3+4+5 = %d\n", res);
    return 0;
}

```

Откомпилируем и запустим:

```
D:\Си>gcc manyargs.c
```

```
D:\Си>a.exe
1+2+3+4+5 = 3
```

Что-то явно не то. Попробуем заменить int на long long (компилируем на Windows, где int и long — синонимы):

```

/* Функция суммирования нескольких целых чисел */
long long sum(long long count, ... )
{
    long long *p = &count + 1;
    long long res = 0;
    for (int i = 0; i < count; ++i) {
        res += *p;
        ++p;
    }
    return res;
}

```

Теперь заработало:

```
D:\Си>gcc manyargs.c
```

```
D:\Си>a.exe
1+2+3+4+5 = 15
```

Но новый вариант на 32-битной платформе работать перестанет.

Как же это сделать переносимо?

В заголовочном файле stdarg.h определён тип va_list и определено несколько

макросов:

- `void va_start (va_list ap, paramN);` — инициализирует `va_list ap`, нужно указать имя последнего явного параметра, ничего не возвращает.
- `type va_arg (va_list ap, type);` — макрос извлекает значение следующего параметра из `ap` (одновременно меняя его) и преобразовывает к указанному типу.
- `void va_end (va_list ap);` — согласно Стандарту, нужно вызывать после завершения перебора всех дополнительных параметров. Считается, что на некоторых платформах макросы `va_start` и `va_arg` видоизменяют фрейм стека, а `va_end` возвращает «всё как было».

Перепишем нашу функцию с использованием этих конструкций:

```
/* Функция суммирования нескольких целых чисел */
int sum(int count, ... )
{
    va_list args;
    va_start(args, count);
    int result = 0;
    for (int i = 0; i < count; ++i) {
        result += va_arg(args, int);
    }
    va_end(args);
    return result;
}
```

и попробуем откомпилировать и запустить:

```
D:\Си>gcc manyargs.c
```

```
D:\Си>a.exe
```

```
1+2+3+4+5 = 15
```

Работает!

Программа будет работать, если поменяем тип на `double` и при вызове запишем вещественные числа:

```
#include <stdarg.h>
#include <stdio.h>

/* Функция суммирования нескольких целых чисел */
double sum(int count, ... )
{
    va_list args;
    va_start(args, count);
    double result = 0;
    for (int i = 0; i < count; ++i) {
        result += va_arg(args, double);
    }
}
```

```

    }
    va_end(args);
    return result;
}

int main()
{
    double res = sum(5, 1.0, 2.0, 3.0, 4.0, 5.0);
    printf("1+2+3+4+5 = %f\n", res);
    return 0;
}

```

Тоже работает:

```
D:\Си>gcc manyargs.c
```

```
D:\Си>a.exe
1+2+3+4+5 = 15.000000
```

Типы передаваемых параметров могут быть разными. Перепишем функцию `sum`, чтобы она первым параметром принимала форматную строку (содержащую буквы `i` и `f`), описывающую типы и количество последующих параметров:

```

#include <assert.h>
#include <stdarg.h>
#include <stdio.h>

/* Функция суммирования нескольких целых чисел */
double sum(const char *fmt, ... )
{
    va_list args;
    va_start(args, fmt);
    double result = 0;
    for (const char *p = fmt; *p != '\0'; ++p) {
        if (*p == 'i') result += va_arg(args, int);
        else if (*p == 'f') result += va_arg(args, double);
        else assert(*p == 'i' || *p == 'f');
    }
    va_end(args);
    return result;
}

int main()
{
    double res = sum("iiffi", 1, 2, 3.3, 4.4, 5);
    printf("1+2+3+4+5 = %f\n", res);
    return 0;
}

```

Здесь мы используем `assert` для обработки случая, когда очередной символ форматной строки не является 'i' или 'f'. Работает:

```
D:\Си>gcc manyargs.c
```

```
D:\Си>a.exe
```

```
1+2+3+4+5 = 15.700000
```

Смотрите в следующей серии

- Передача `va_list` в другие функции, функции `vprintf`, `vscanf`.
- Написание простой библиотеки модульного тестирования на языке Си.